
Analisis Jenis Produk dan Visibilitas Ruang Pajang Terhadap Penjualan Ritel



INTRODUCTION

Self-Overview

**Raden Maulana
Akbar**

- Education
 - SMAN 7 Bekasi
- Project
 - Employee Attrition & High-Performance Loss Analysis



Problem Statement



Analisis Jenis Produk dan Visibilitas Ruang Pajang Terhadap Penjualan Ritel

Ruang pajang (rak) toko ritel yang terbatas menuntut manajemen untuk menata produk secara strategis guna mendongkrak omzet. Tantangan utamanya adalah memastikan karakteristik dan tingkat visibilitas produk di rak benar-benar menghasilkan penjualan yang optimal, bukan sekadar membuang ruang.

Data Exploration



Sumber: BigMart Sales Data.csv

Ukuran: 8523 baris × 12 kolom

Isi dari Raw Data

- Outlet (Outlet_Identifier, Outlet_Establishment_Year, Outlet_Size, Outlet_Location_Type, Outlet_Type, Item_Outlet_Sales)
- Data Produk (Item_Identifier, Item_Weight, Item_Fat_Content, Item_Visibility, Item_Type, Item_MRP)

```
df = pd.read_csv('BigMart Sales Data.csv')
df
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type	Outlet_Type	Item_Outlet_Sales
0	FDA15	9.300	Low Fat	0.016047	Dairy	249.8092	OUT049	1999	Medium	Tier 1	Supermarket Type1	3735.1380
1	DRC01	5.920	Regular	0.019278	Soft Drinks	48.2692	OUT018	2009	Medium	Tier 3	Supermarket Type2	443.4228
2	FDN15	17.500	Low Fat	0.016760	Meat	141.6180	OUT049	1999	Medium	Tier 1	Supermarket Type1	2097.2700
3	FDX07	19.200	Regular	0.000000	Fruits and Vegetables	182.0950	OUT010	1998	NaN	Tier 3	Grocery Store	732.3800
4	NCD19	8.930	Low Fat	0.000000	Household	53.8614	OUT013	1987	High	Tier 3	Supermarket Type1	994.7052

Data Exploration

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Item_Identifier        8523 non-null   str
1   Item_Weight            7060 non-null   float64
2   Item_Fat_Content       8523 non-null   str
3   Item_Visibility        8523 non-null   float64
4   Item_Type              8523 non-null   str
5   Item_MRP               8523 non-null   float64
6   Outlet_Identifier      8523 non-null   str
7   Outlet_Establishment_Year 8523 non-null   int64
8   Outlet_Size            6113 non-null   str
9   Outlet_Location_Type   8523 non-null   str
10  Outlet_Type            8523 non-null   str
11  Item_Outlet_Sales      8523 non-null   float64
dtypes: float64(4), int64(1), str(7)
```

```
#cek duplikat setiap baris
duplikat_jml = df.duplicated().sum()
duplikat_persen = (duplikat_jml / len(df)) * 100
print(f"Jumlah baris duplikat: {duplikat_jml} ({duplikat_persen:.2f}%)")
```

Jumlah baris duplikat: 0 (0.00%)

```
#cek persentase missing value
df_miss = pd.DataFrame({
    'data_hilang' : df.isna().sum(),
    'Percentage' : (df.isna().mean()*100).round(2)
})
df_miss = df_miss[df_miss['data_hilang'] > 0]
df_miss
```

	data_hilang	Percentage
Item_Weight	1463	17.17
Outlet_Size	2410	28.28

```
df.describe()
```

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Item_Outlet_Sales
count	7060.000000	8523.000000	8523.000000	8523.000000	8523.000000
mean	12.857645	0.066132	140.992782	1997.831867	2181.288914
std	4.643456	0.051598	62.275067	8.371760	1706.499616
min	4.555000	0.000000	31.290000	1985.000000	33.290000
25%	8.773750	0.026989	93.826500	1987.000000	834.247400
50%	12.600000	0.053931	143.012800	1999.000000	1794.331000
75%	16.850000	0.094585	185.643700	2004.000000	3101.296400
max	21.350000	0.328391	266.888400	2009.000000	13086.964800

Hasil Pengamatan

Missing Value: Ditemukan pada Item_Weight (17,17%) dan Outlet_Size (28,28%) yang memerlukan tindakan imputasi.

Anomali Visibilitas: Nilai minimum Item_Visibility bernilai 0.0 (perlu diperbaiki karena barang yang tidak terpajang di rak tidak mungkin terjual).

Duplikasi Data: 0% (tidak ditemukan data duplikat).

Data Exploration

```
#checking nilai kolom yang unik
cat_cols = df.select_dtypes(include=['object', 'string', 'category', 'bool'])

df_ceksdf = pd.DataFrame([[
    'column': col,
    'unique_values': cat_cols[col].dropna().unique().tolist(),
    'n_unique': cat_cols[col].nunique(dropna=True)
] for col in cat_cols.columns])

pd.set_option('display.max_colwidth', None)
df_ceksdf
```

	column	unique_values	n_unique
0	Item_Identifier	[FDA15, DRC01, FDN15, FDX07, NCD19, FDP36, FDO10, FDP10, FDH17, FDU28, FDY07, FDA03, FDX32, FDS46, FDF32, FDP49, NCB42, DRI11, FDU02, FDN22, FDW12, NCB30, FDC37, FDR28, NCD06, FDV10, DRJ59, FDE51, FDC14, FDV38, NCS17, FDP33, FDO23, DRH01, NCX29, FDV20, DRZ11, FDX10, FDB34, FDK43, FDA46, FDC02, FDL50, FDM39, NCP05, FDV49, FDL12, FDS02, NCL17, FDM40, FDR13, FDA43, NCP18, FDK21, NCX54, DRK35, FDY21, FDI26, FDM20, FDV27, FDF09, FDY40, FDY45, FDC46, FDH19, FDZ03, DRH37, NCI17, FDJ58, FDH35, FDG02, NCZ18, FDC29, FDQ10, FDN48, FDL04, FDV25, FDD58, FDN04, FDV45, NCL18, FDR12, FDG20, FDZ55, FDQ49, FDN33, FDN27, FDW20, DRG27, DRI25, FDA44, NCR17, FDU04, FDF41, FDB56, FDT28, FDD10, FDW57, DRB48, FDP09, ...]	1559
1	Item_Fat_Content	[Low Fat, Regular, low fat, LF, reg]	5
2	Item_Type	[Dairy, Soft Drinks, Meat, Fruits and Vegetables, Household, Baking Goods, Snack Foods, Frozen Foods, Breakfast, Health and Hygiene, Hard Drinks, Canned, Breads, Starchy Foods, Others, Seafood]	16
3	Outlet_Identifier	[OUT049, OUT018, OUT010, OUT013, OUT027, OUT045, OUT017, OUT046, OUT035, OUT019]	10
4	Outlet_Size	[Medium, High, Small]	3
5	Outlet_Location_Type	[Tier 1, Tier 3, Tier 2]	3
6	Outlet_Type	[Supermarket Type1, Supermarket Type2, Grocery Store, Supermarket Type3]	4

```
#checking kolom yang mewakili nilai setiap baris
for col in df.columns:
    jumlah_unik = df[col].nunique()
    print(f"Kolom {col}: {jumlah_unik} nilai unik")
```

Kolom Item_Identifier: 1559 nilai unik
Kolom Item_Weight: 415 nilai unik
Kolom Item_Fat_Content: 5 nilai unik
Kolom Item_Visibility: 7880 nilai unik
Kolom Item_Type: 16 nilai unik
Kolom Item_MRP: 5938 nilai unik
Kolom Outlet_Identifier: 10 nilai unik
Kolom Outlet_Establishment_Year: 9 nilai unik
Kolom Outlet_Size: 3 nilai unik
Kolom Outlet_Location_Type: 3 nilai unik
Kolom Outlet_Type: 4 nilai unik
Kolom Item_Outlet_Sales: 3493 nilai unik

Hasil eksplorasi dari checking nilai unik setiap kolom

Tidak ditemukan kolom yang unik untuk mewakili setiap baris

```
# Export hasil nilai unik per kolom untuk dianalisa oleh AI memeriksa nilai yang tidak konsisten
df_ceksdf.to_csv('nilai unik.csv', index=False)
```

Hasil Analisa AI

terdapat inkonsistensi nilai pada kolom Item_Fat_Content dan nilai pada kolom lain sudah konsisten (tidak redundan)

Hasil Pengamatan

Tidak ditemukan satu kolom tunggal yang unik untuk mewakili setiap baris data.

Ditemukan redundansi penulisan pada kolom Item_Fat_Content

Kolom-kolom kategorikal lainnya (Item_Type, Outlet_Identifier, Outlet_Size, Outlet_Location_Type, dan Outlet_Type) sudah konsisten dan tidak memiliki penulisan yang redundan

Data Cleaning

1. Cek Inkonsistensi Data

```
# cek inkonsistensi nilai pada kolom Item_Fat_Content
df['Item_Fat_Content'].unique()
```

```
<ArrowStringArray>
['Low Fat', 'Regular', 'low fat', 'LF', 'reg']
Length: 5, dtype: str
```

```
# aksi untuk menyeragamkan inkonsistensi data Item_Fat_Content
df_clean['Item_Fat_Content'] = df['Item_Fat_Content'].replace({
    'low fat': 'Low Fat',
    'LF': 'Low Fat',
    'reg': 'Regular'
})
df_clean['Item_Fat_Content'].unique()
```

```
<ArrowStringArray>
['Low Fat', 'Regular']
Length: 2, dtype: str
```

2. Penanganan Nilai Anomali

```
# Memfilter dataframe untuk melihat baris yang nilai Item_Visibility-nya sama dengan 0
zero_visibility_rows = df[df['Item_Visibility'] == 0]

# Menghitung jumlah baris yang Item_Visibility-nya 0.0
jml_zero = (df['Item_Visibility'] == 0).sum()
persen_zero = (df['Item_Visibility'] == 0).mean() * 100

print(f"Jumlah baris dengan Item_Visibility = 0.0 : {jml_zero} baris")
print(f"Persentase data visibilitas 0.0 : {persen_zero:.2f}%")

# Menampilkan beberapa baris awal beserta Outlet_Identifier-nya
zero_visibility_rows[['Outlet_Identifier', 'Item_Identifier', 'Item_Visibility', 'Item_Outlet_Sales']].head(5)

Jumlah baris dengan Item_Visibility = 0.0 : 526 baris
Persentase data visibilitas 0.0 : 6.17%
```

	Outlet_Identifier	Item_Identifier	Item_Visibility	Item_Outlet_Sales
3	OUT010	FDX07	0.0	732.3800
4	OUT013	NCD19	0.0	994.7052
5	OUT018	FDP36	0.0	556.6088
10	OUT049	FDY07	0.0	1516.0266
32	OUT018	FDP33	0.0	3068.0064

```
# 1. Ubah nilai 0.0 menjadi NaN terlebih dahulu agar terbaca sebagai missing value
df_clean['Item_Visibility'] = df_clean['Item_Visibility'].replace(0, np.nan)

# 2. Imputasi nilai NaN menggunakan rata-rata (mean) dari tiap Outlet_Identifier yang sama
df_clean['Item_Visibility'] = df_clean['Item_Visibility'].fillna(
    df_clean.groupby('Outlet_Identifier')['Item_Visibility'].transform('mean')
)

# Cek kembali jumlah missing value pada kolom Outlet_Size setelah diimputasi
print("Sisa missing value pada Item_Visibility:", df_clean['Item_Visibility'].isna().sum())
df_clean[['Outlet_Identifier', 'Item_Identifier', 'Item_Visibility', 'Item_Outlet_Sales']].head(5)

Sisa missing value pada Item_Visibility: 0
```

Hasil Pengamatan

Penyeragaman Kategori: Berhasil merapikan lima variasi penulisan pada kolom Item_Fat_Content (Low Fat, Regular, low fat, LF, reg) menjadi dua kategori standar (Low Fat dan Regular).

Penanganan Anomali Visibilitas: Ditemukan 526 baris (6,17%) anomali Item_Visibility bernilai 0.0 (barang menghasilkan penjualan tetapi keterlihatan rak 0). Seluruhnya berhasil diimputasi menggunakan nilai rata-rata visibilitas dari masing-masing Outlet_Identifier.

Validasi Akhir: Kolom Item_Fat_Content kini 100% konsisten dan sisa anomali pada Item_Visibility berhasil menjadi 0 baris.

Data Cleaning

3. Penanganan Missing Value pada kolom Item_Weight

```
# Cek konsistensi berat pada kolom Item_Weight
weight_check = df_clean.groupby('Item_Identifier')['Item_Weight'].nunique()
print(f"Jumlah produk dengan variasi berat > 1: {(weight_check > 1).sum()}")
```

```
#menampilkan hasil data dengan Item_Identifier menghasilkan berat yang sama
sample_id = df_clean[df_clean['Item_Weight'].isna()][['Item_Identifier']].iloc[2]
print(f"\nContoh perbandingan untuk Item_Identifier '{sample_id}':")
```

```
df_clean[df_clean['Item_Identifier'] == sample_id][['Item_Identifier', 'Item_Weight', 'Outlet_Identifier']]
```

Jumlah produk dengan variasi berat > 1: 0

Contoh perbandingan untuk Item_Identifier 'FDW12':

	Item_Identifier	Item_Weight	Outlet_Identifier
21	FDW12	NaN	OUT027
1788	FDW12	8.315	OUT010
2973	FDW12	8.315	OUT049

```
df_clean[df_clean['Item_Identifier'].isin(['FDN52', 'FDK57', 'FDE52', 'FDQ60'])
[ 'Item_Identifier', 'Item_Weight', 'Outlet_Identifier', 'Item_Outlet_Sales' ]
]
```

	Item_Identifier	Item_Weight	Outlet_Identifier	Item_Outlet_Sales
927	FDN52	NaN	OUT027	1569.9564
1922	FDK57	NaN	OUT027	4434.2280
4187	FDE52	NaN	OUT027	3453.5046
5022	FDQ60	NaN	OUT019	120.5098

```
# Aksi imputasi Item_Weight menggunakan nilai rata-rata dari Item_Identifier yang sama
df_clean['Item_Weight'] = df_clean.groupby('Item_Identifier')['Item_Weight'].transform(lambda x: x.fillna(x.mean()))
```

```
# Cek kembali jumlah missing value pada kolom Item_Weight setelah diimputasi
print("Sisa missing value pada Item_Weight:", df_clean['Item_Weight'].isna().sum())
```

Sisa missing value pada Item_Weight: 4

```
#Imputasi 4 nilai yang kosong dengan rata-rata kolom Item_Weight
df_clean['Item_Weight'] = df_clean['Item_Weight'].fillna(df_clean['Item_Weight'].mean())
```

```
# Cek kembali jumlah missing value pada kolom Item_Weight setelah diimputasi
print("Sisa missing value pada Item_Weight:", df_clean['Item_Weight'].isna().sum())
```

Sisa missing value pada Item_Weight: 0

Hasil Cleaning

Imputasi Tahap 1 (Berdasarkan ID Produk): Sebanyak 1.459 missing value pada Item_Weight berhasil diisi menggunakan rata-rata berat dari Item_Identifier yang sama, karena terbukti setiap produk memiliki berat yang konsisten (berat = 0).

Imputasi Tahap 2 (Penanganan Sisa 4 Data): Sisa 4 data yang masih kosong (karena produk tersebut berdiri sendiri dan tidak memiliki pembanding ID lain) diimputasi menggunakan nilai rata-rata keseluruhan (overall mean) untuk mempertahankan data Item_Outlet_Sales.

Validasi Akhir: Sisa missing value pada kolom Item_Weight berhasil menjadi 0 baris (100% bersih).

Data Cleaning

4. Penanganan Missing Value pada kolom Outlet_Size

```
#Uji hubungan antar 2 kolom
pd.crosstab(df_clean['Outlet_Type'], df_clean['Outlet_Size'], dropna=False)
```

	Outlet_Size	High	Medium	Small	NaN
Outlet_Type					
Grocery Store		0	0	528	555
Supermarket Type1		932	930	1860	1855
Supermarket Type2		0	928	0	0
Supermarket Type3		0	935	0	0

```
# uji Lebih Lanjut untuk nilai yang hilang dengan tipe Supermarket Type1 dengan kolom Outlet_Location_Type
sup1_df = df_clean[df_clean['Outlet_Type'] == 'Supermarket Type1']

# Crosstab antara Outlet_Location_Type dan Outlet_Size= Supermarket Type1
pd.crosstab(sup1_df['Outlet_Location_Type'], sup1_df['Outlet_Size'], dropna=False)
```

	Outlet_Size	High	Medium	Small	NaN
Outlet_Location_Type					
Tier 1		0	930	930	0
Tier 2		0	0	930	1855
Tier 3		932	0	0	0

```
# Imputasi nilai yang kosong pada Outlet_Size
df_clean['Outlet_Size'] = df_clean['Outlet_Size'].fillna('Small')
# Cek kembali jumlah missing value pada kolom Outlet_Size setelah diimputasi
print("Sisa missing value pada Outlet_Size:", df_clean['Outlet_Size'].isna().sum())

Sisa missing value pada Outlet_Size: 0

#cek apakah data yang di imputasi terisi dengan benar
pd.crosstab(df_clean['Outlet_Type'], df_clean['Outlet_Size'], dropna=False)
```

	Outlet_Size	High	Medium	Small
Outlet_Type				
Grocery Store		0	0	1083
Supermarket Type1		932	930	3715
Supermarket Type2		0	928	0
Supermarket Type3		0	935	0

Hasil Cleaning

Analisis Crosstab: 2.410 missing value Outlet_Size berasal dari kelompok Grocery Store (555) dan Supermarket Type1 Tier 2 (1.855).

Dasar Imputasi: Mayoritas data pembanding pada kelompok tersebut berukuran Small, sehingga nilai kosong diimputasi dengan kategori 'Small'.

Hasil Akhir: Seluruh data terisi presisi dan sisa missing value pada Outlet_Size berhasil menjadi 0 baris.

Data Cleaning

5. Mengidentifikasi Outlier dan Menangani Outlier

```
'''cek Outlier dengan IQR
'''

# Cek outlier untuk kolom Item_Outlet_Sales
Q1 = df_clean['Item_Outlet_Sales'].quantile(0.25)
Q3 = df_clean['Item_Outlet_Sales'].quantile(0.75)

IQR = Q3 - Q1

batas_bawah = Q1 - (1.5 * IQR)
batas_atas = Q3 + (1.5 * IQR)

print(f"Batas Bawah: {batas_bawah}")
print(f"Batas Atas: {batas_atas}")

outlier_bawah = df_clean[df_clean['Item_Outlet_Sales'] < batas_bawah]
outlier_atas = df_clean[df_clean['Item_Outlet_Sales'] > batas_atas]
print(f"Jumlah baris di bawah batas normal: {len(outlier_bawah)} baris")
print(f"Jumlah baris data dengan penjualan di atas batas normal (outlier): {len(outlier_atas)} baris")

Batas Bawah: -2566.3261
Batas Atas: 6501.8699
Jumlah baris di bawah batas normal: 0 baris
Jumlah baris data dengan penjualan di atas batas normal (outlier): 186 baris
```

```
# Cek outlier untuk kolom Item_Visibility
Q1_vis = df_clean['Item_Visibility'].quantile(0.25)
Q3_vis = df_clean['Item_Visibility'].quantile(0.75)
IQR_vis = Q3_vis - Q1_vis
batas_atas_vis = Q3_vis + (1.5 * IQR_vis)

print(f"Batas Atas Item_Visibility: {batas_atas_vis}")
outlier_visibilitas_tinggi = df_clean[df_clean['Item_Visibility'] > batas_atas_vis]
print(f"Jumlah baris data dengan Item_Visibility di atas batas normal: {len(outlier_visibilitas_tinggi)} baris")

Batas Atas Item_Visibility: 0.19157102725000003
Jumlah baris data dengan Item_Visibility di atas batas normal: 154 baris

.. .. -
```

Hasil Cleaning

Outlier Penjualan (Item_Outlet_Sales): Teridentifikasi 186 baris data di atas batas normal IQR (> 6.501,87).

Outlier Visibilitas (Item_Visibility): Teridentifikasi 154 baris data di atas batas normal IQR (> 0,1916).

Keputusan Penanganan: Seluruh outlier dipertahankan (tidak dihapus) karena mewakili kondisi bisnis nyata (high-performing products/outlets), bukan disebabkan oleh kesalahan input data (data entry error).

Data Manipulation

1. Membuat Dataset

```
#buat dataframe baru untuk membuat analisa
kolom_cek = ['Item_Type', 'Item_Visibility', 'Item_Outlet_Sales']

df_analysis = df_clean[kolom_cek].copy()

''' Membulatkan nilai Item_Outlet_Sales menjadi 2 angka di belakang koma
    Membulatkan nilai Item_Visibility menjadi 4 angka di belakang koma
    ...
df_analysis['Item_Outlet_Sales'] = df_analysis['Item_Outlet_Sales'].round(2)
df_analysis['Item_Visibility'] = df_analysis['Item_Visibility'].round(4)

df_analysis.head()
```

	Item_Type	Item_Visibility	Item_Outlet_Sales
0	Dairy	0.0160	3735.14
1	Soft Drinks	0.0193	443.42
2	Meat	0.0168	2097.27
3	Fruits and Vegetables	0.1071	732.38
4	Household	0.0640	994.71

2. Filter Data

```
# membuat filtering untuk Memilih produk dengan penjualan di atas rata-rata keseluruhan
rata_penjualan = df_analysis['Item_Outlet_Sales'].mean()
df_filter = df_analysis[df_analysis['Item_Outlet_Sales'] > rata_penjualan]

print(f"Rata-rata acuan penjualan: {rata_penjualan:.2f}")
print(f"Jumlah baris setelah difilter (penjualan di atas rata-rata): {len(df_filter)}")
df_filter.head()
```

Rata-rata acuan penjualan: 2181.29

Jumlah baris setelah difilter (penjualan di atas rata-rata): 3523

	Item_Type	Item_Visibility	Item_Outlet_Sales
0	Dairy	0.0160	3735.14
7	Snack Foods	0.1275	4022.76
9	Frozen Foods	0.0944	4710.54
11	Dairy	0.0455	2187.15
18	Hard Drinks	0.0342	2303.67

Data Manipulation

3. Grouping dan agregasi

```
# 1. Melakukan Grouping berdasarkan Item_Type dan Agregasi
df_group = df_analysis.groupby('Item_Type').agg(
    Jumlah_Produk=('Item_Outlet_Sales', 'count'),
    Total_Penjualan=('Item_Outlet_Sales', 'sum'),
    Rata_Rata_Penjualan=('Item_Outlet_Sales', 'mean'),
    Rata_Rata_Visibilitas=('Item_Visibility', 'mean')
).reset_index()

# 2. Melakukan Sorting (Mengurutkan berdasarkan Rata-Rata Penjualan tertinggi ke terendah)
df_group_sort = df_group.sort_values(by='Rata_Rata_Penjualan', ascending=False).round(2)
df_group_sort.head(2)
```

	Item_Type	Jumlah_Produk	Total_Penjualan	Rata_Rata_Penjualan	Rata_Rata_Visibilitas
15	Starchy Foods	148	351401.26	2374.33	0.07
12	Seafood	64	148868.17	2326.07	0.08

```
# melakukan sorting (Mengurutkan berdasarkan Produk yang terjual tertinggi ke terendah)
df_group_sort2 = df_group.sort_values(by='Jumlah_Produk', ascending=False).round(2)
df_group_sort2.head(2)
```

	Item_Type	Jumlah_Produk	Total_Penjualan	Rata_Rata_Penjualan	Rata_Rata_Visibilitas
6	Fruits and Vegetables	1232	2820059.87	2289.01	0.07
13	Snack Foods	1200	2732786.09	2277.32	0.07

Hasil Analisa



Dominasi Omzet Penjualan

Jenis produk seperti Fruits and Vegetables dan Snack Foods menjadi penyumbang total omzet terbesar bagi ritel karena memiliki jumlah varian produk yang paling masif di toko (di atas 1.200 produk).



Performa per Produk (High Value)

Meskipun jumlah produknya lebih sedikit, jenis produk Starchy Foods (2374.33) dan Seafood (2326.07) mencatatkan rata-rata penjualan per produk tertinggi dibanding jenis produk lainnya



Dinamika Visibilitas Ruang Pajang (Rak)

jenis produk Breakfast menduduki peringkat teratas dalam hal rata-rata visibilitas rak (0.09), namun penjualannya berada di papan tengah. Ini menunjukkan bahwa ruang pajang yang luas tidak otomatis menaikkan angka penjuala tanpa dukungan permintaan pasar yang tinggi

Rekomendasi Bisnis



Optimalisasi Ulang Ruang Etalase

Segera evaluasi dan kurangi jatah rak premium untuk produk Breakfast. Visibilitas luas nyatanya tidak mengerek penjualan mereka. Ruang pajang berharga ini sebaiknya dilimpahkan ke kategori dengan perputaran stabil.



Strategi Promosi Kategori Unggulan

Kategori Starchy Foods dan Seafood terbukti ampuh menghasilkan margin rata-rata yang kuat. Terapkan strategi bundling produk dan tempatkan di etalase strategis guna memancing keranjang belanja konsumen yang lebih besar.